

Occasio - Event Management and Booking System

Interview Preparation — Technical Deep Dive

Date: May 14, 2026

Prepared from direct inspection of the repository: **README**, **QUICKSTART**, **package files and lockfiles**, **environment examples**, **Prisma schema**, **migrations**, **backend source**, **frontend source**, **tests**, **deployment files**, and the existing project documentation PDF.

Table of Contents

1. Project Overview	3
2. Full Tech Stack Breakdown	4
3. System Architecture	6
4. Key Technical Decisions	9
5. Deep Dive - Hardest Problems	11
7. Interview Questions and Answers	14
8. Follow-up Questions	17
9. Quick Reference Card	19

1. Project Overview

Occasio is a full-stack **event management and booking platform**. It lets organizers create and publish events, design custom registration forms, collect online payments, issue secure QR-coded tickets, run venue check-in, manage team access, view analytics, send attendee communications, and generate certificates. Public users can browse events, register, pay through **Razorpay** or **PhonePe**, download ticket PDFs, add calendar files, join waitlists, vote in polls, and leave reviews.

The real-world problem is the gap between a marketing-style event page and the operational work required after someone registers: payment correctness, ticket generation, email delivery, scanner reliability, attendee capacity, discount usage, check-in state, certificates, and team delegation. The codebase treats the event as a full lifecycle workflow rather than only a listing page.

The current implementation goes well beyond the original scaffold PDF. The original PDF described a Wix Events-style platform with **React**, **Node.js**, **PostgreSQL**, **AJV**, **Razorpay/Stripe**, **QR codes**, background jobs, and S3-like storage. The repository now includes those ideas plus **Prisma 6**, **PhonePe**, **Cloudflare R2**, **Cloudinary fallback**, **PDFKit**, **pdf-lib**, **BullMQ**, **team roles**, **polls**, **reviews**, **push subscriptions**, **ticket tiers**, **speakers**, and certificate workflows.

Domain And Use Case

- **Organizer workflow:** sign in, create an event, upload posters, configure ticket tiers, build registration fields, publish, monitor registrations, issue certificates, and manage team access.
- **Attendee workflow:** discover an event, complete a dynamic form, choose a ticket tier, apply a discount, pay or RSVP, receive a ticket PDF, and scan the QR code at the venue.
- **Venue workflow:** staff use camera scanning or manual ticket entry, the backend verifies the ticket signature and event access, and the ticket is atomically marked checked in or checked out.
- **Post-event workflow:** organizers review analytics, send certificates to checked-in users, analyze revenue, and gather reviews or poll results.

Key Outcomes And Metrics Found In Code

- Analytics endpoints compute **total registrations**, **paid registrations**, **pending registrations**, **revenue**, **average order value**, **conversion rate**, **capacity used**, **check-in rate**, **discount usage**, **tier breakdown**, **payment provider breakdown**, daily timelines, hourly distribution, and recent registrations.
- Ticket operations track **checkedIn**, **checkedInAt**, **checkedOutAt**, **revoked**, **validUntil**, PDF URL, QR payload, and order/registration links.
- Payment completion updates registration state, order state, discount usage, ticket tier sold counts, ticket generation, email sending, and SMS notification from one canonical service.
- The frontend exposes separate admin, public, scanner, team member, analytics, discounts, financial, certificate, and form-builder surfaces through **React Router**.

2. Full Tech Stack Breakdown

The stack below is read from source files, package files, lockfiles, configuration, and route implementations. Where a reason is stated, it is inferred from how the tool is used in the codebase rather than guessed from marketing claims.

Frontend runtime	React 18 with Vite 8 . The app uses lazy-loaded route pages, a single-page app rewrite on Vercel, and a Vite dev proxy for backend routes. This is a fast SPA choice for a dashboard-heavy product where the browser owns form state, scanner state, PDF preview state, and route-level screens.
Frontend UI	Tailwind CSS , custom CSS layers, Outfit font, lucide-react icons, motion , and a Three.js shader background component. Tailwind and small component primitives are used instead of a large component framework, which keeps the UI flexible for admin dashboards, glass-card panels, dock navigation, and public event pages.
Frontend routing and data	react-router-dom 6 , Axios , React Context , react-hook-form , react-hot-toast , and date-fns . Axios centralizes the API base URL and JWT injection; contexts handle auth and theme; react-hook-form keeps dynamic registration forms manageable.
Scanner and PDF UI	html5-qrcode , jsQR , react-pdf , and pdfjs-dist . These libraries serve camera scanning, QR parsing, and authenticated certificate-template preview in the browser.
PWA	vite-plugin-pwa and service worker registration are present but gated by VITE_ENABLE_PWA . This makes offline/push features opt-in so development and unsupported deployments do not break.
Backend runtime	Node.js 18+ , Express 4 , and native ES modules . Express fits the route-heavy REST API surface and keeps payment webhooks, upload middleware, and scanner endpoints explicit.
Database and ORM	PostgreSQL through Prisma 6.19 . The schema uses enums, JSON fields, relational constraints, indexes, and migrations. The database config is hardened for managed Postgres/Neon style pooled connections with SSL and startup retry.
Auth and security	jsonwebtoken , bcryptjs , helmet , cors , express-rate-limit , role middleware, event-access middleware, and HMAC QR signatures. The design is stateless and deployment-friendly while still applying route-level ownership and team checks.
Validation	express-validator validates auth inputs; AJV and ajv-formats validate dynamic registration schemas; route-specific checks validate tier, discount, payment, event, certificate, and check-in state.
Payments	Razorpay , PhonePe , and a Stripe enum/stub. Razorpay order creation/signature verification and raw webhook HMAC verification are implemented. PhonePe redirect/status verification is implemented with sandbox defaults and production env support.

Tickets and documents	qrcode , PDFKit , pdf-lib , and ics . PDFKit creates ticket PDFs; pdf-lib overlays certificate fields on uploaded PDF templates; qrcode generates signed ticket QR images; ics creates calendar downloads.
Storage	Cloudflare R2 via @aws-sdk/client-s3 , optional generic S3 , Cloudinary , and local uploads. R2 is the preferred private PDF/document path, Cloudinary is a compatibility fallback, and local disk keeps development usable.
Async and messaging	BullMQ with ioredis when Redis is available; fallback setImmediate background work when Redis is absent. Nodemailer , Twilio , and web-push serve outbound email, SMS, WhatsApp-ready notification scaffolding, and push subscriptions.
Deployment and tooling	Azure App Service workflow for the backend, Vercel config for the frontend, Prisma migrations , npm scripts , Node test runner , and startup scripts. This matches the split nature of a public SPA and API service.

Libraries Present But Limited Or Optional

- **Stripe** is represented in enums and a placeholder webhook route, but the working payment flows are **Razorpay** and **PhonePe**.
- **passkit-generator** is dynamically imported by the wallet service, but it is not listed in package.json, so Apple Wallet support is currently an optional placeholder path unless the dependency and certificates are added.
- **Redis** and **BullMQ** are production-oriented enhancements. The code intentionally keeps a no-Redis fallback path for local development and simple deployments.
- There are no **ML** or **AI** model files, training scripts, inference services, vector stores, or evaluation metrics in this repository.

3. System Architecture

High-Level Component Model

- **Frontend SPA:** React/Vite application with public pages, protected admin pages, team pages, scanner pages, certificate tooling, and shared components. It calls the backend through an Axios instance that injects a JWT from localStorage.
- **Express API:** central server with route modules for auth, events, admin, registrations, webhooks, tickets, discounts, reviews, polls, push, team access, feature endpoints, wallet endpoints, and waitlists.
- **Database layer:** Prisma models represent users, events, forms, registrations, orders, tickets, waitlists, discounts, reviews, polls, notifications, team members, push subscriptions, ticket tiers, speakers, and event reminders.
- **Storage layer:** posters, certificate templates, and ticket PDFs can move through local uploads, Cloudinary, generic S3, or Cloudflare R2. PDFs prefer private object storage while public images can be hosted through Cloudinary/S3/local paths.
- **External services:** Razorpay and PhonePe process payments, SMTP sends emails, Twilio sends booking notifications, web-push handles browser push, and Redis/BullMQ optionally processes ticket generation and email jobs.

Registration-To-Ticket Data Flow

- 1 The public event page loads event detail, published status, ticket tiers, speakers, reviews, polls, waitlist state, and form schema through public endpoints.
- 2 The registration page renders fields from the stored **Form.schema**. If no custom form exists, the backend provides a default attendee form with name, email, and phone fields.
- 3 On submit, the backend verifies event availability, tier selection, discount code constraints, capacity constraints, and validates the dynamic form with **AJV**.
- 4 The registration route creates a **Registration** and an **Order**. Free or RSVP events complete immediately; paid events create a Razorpay order or PhonePe redirect.
- 5 Payment verification can arrive through frontend callback verification or a raw-body webhook. The canonical **order-completion service** then runs a serializable transaction that updates order, registration, tier sold counts, discount usage, and ticket generation state.
- 6 Ticket generation creates or reuses a ticket, signs a QR payload with HMAC, renders a branded PDF using **PDFKit**, stores it in R2/Cloudinary/local storage, updates the ticket record, and sends email/SMS through a queue or fallback background task.

Check-In Data Flow

- 1 Staff scan a QR code with **html5-qrcode** or enter a ticket manually. The scanner submits a flexible payload to **POST /api/tickets/verify**.
- 2 The backend parser accepts raw IDs, JSON strings, URL-encoded JSON, base64 JSON, URL query strings, and structured QR objects. This is important because QR scanners and ticket screenshots can return different formats.
- 3 The server checks ticket existence, event access, revocation, payment/registration status, QR signature validity, and duplicate check-in state.

- 4 Check-in and check-out operations use atomic database updates such as **updateMany** with state predicates, which avoids race conditions when the same ticket is scanned twice.
- 5 Admin and team dashboards refresh attendees and check-in stats so venue staff can see total tickets, checked-in count, currently-inside count, and pending attendees.

Database Schema And Relationships

User	Organizer/admin identity with role ; owns events and can belong to teams by email.
Event	Central aggregate. Links to organizer, form, registrations, waitlist, discounts, reviews, polls, team members, ticket tiers, speakers, and reminders. Stores type, category, ticket style, certificate settings, location, capacity, price, tags, publication state, and poster/template URLs.
Form	One-to-one event form stored as JSONB ; drives dynamic frontend rendering and AJV backend validation.
Registration	Attendee submission with form response JSON, attendee contact fields, status, price snapshot, discount snapshot, and relations to orders/tickets.
Order	Payment intent and result with provider enum, provider order/payment IDs, amount, currency, status, and paymentData JSON.
Ticket	Issued ticket tied to an order and event, with QR payload, PDF URL, revoked/valid-until state, and check-in/check-out timestamps.
TicketTier	Per-event ticket product with price, capacity, sold count, sales window, sort order, and active flag.
DiscountCode	Per-event code with percentage/fixed type, max uses, used count, date range, and active flag.
TeamMember	Event-level role delegation with SUPER_MANAGER, MANAGER, SCANNER, STAFF, acceptedAt, and invitation email.
Poll, Review, Waitlist, PushSubscription, Speaker, EventReminder	Engagement, operations, and lifecycle support models that attach to events and attendees.

API Design

- **/api/auth**: register and login with password hashing and JWT issuance.
- **/api/events**: public event listing, categories, event detail by ID or slug, form schema, tiers, speakers, and calendar export.
- **/api/admin**: organizer event CRUD, poster/template upload, form builder, registrations, analytics, broadcasts, financials, check-in management, team management, certificate generation, ticket style, and feature configuration.
- **/api/events/:id/register** and **/api/orders/:id/***: registration, payment session creation, Razorpay verification, and PhonePe verification.

- **/api/webhooks**: raw request body payment webhooks with signature verification.
- **/api/tickets**: scanner verification and ticket PDF download routes.
- **/api/team**: invited team member event list, event detail, attendee list, check-in stats, accept invitation, check-in, and check-out.
- **/api/discounts**, **/api/reviews**, **/api/polls**, **/api/waitlist**, **/api/push**, and **/api/wallet**: supporting event-commerce and engagement surfaces.

Caching, Async, Streaming, And Event-Driven Patterns

- **BullMQ** queues are used for ticket generation and email sending when Redis is configured and reachable.
- When Redis is unavailable, ticket creation and communication are scheduled through **setImmediate**, keeping the API responsive without making Redis mandatory.
- Payment webhooks act as the external event source for final payment confirmation. The backend uses raw body parsing for webhook integrity.
- The frontend uses lazy route imports and stateful screens rather than server-side rendering. Push subscription endpoints exist for future event notifications.
- PDF and object retrieval can stream from R2 or redirect to signed/local URLs depending on where the artifact was stored.

4. Key Technical Decisions

Split SPA plus API

The repository separates **frontend** and **backend**. This fits a product with a rich browser UI, camera scanner, PDF preview, and admin workflows while letting the backend run on Azure App Service and the frontend run on Vercel. The trade-off is cross-origin configuration, duplicated environment setup, and local dev needing two servers.

Prisma with PostgreSQL

Prisma gives a typed relational model and migration history for a domain with many relationships: events, users, forms, registrations, orders, tickets, teams, polls, reminders, and discounts. PostgreSQL JSONB supports custom form schemas and responses. The trade-off is migration discipline and needing to understand Prisma transaction errors such as P2034.

Dynamic forms as JSON schema

Forms are not hardcoded per event. Admins define fields and the frontend renders them dynamically while the backend validates with **AJV**. This is more flexible than static forms, but it creates validation and analytics complexity because responses are semi-structured.

Payment completion as a service

The code funnels Razorpay verification, PhonePe verification, free/RSVP completion, and webhook completion into **completePaidOrder**. That reduces duplicated payment-state logic and is the correct place to handle idempotency, ticket tiers, discounts, registration status, and queue work.

Serializable transaction for correctness

Order completion runs inside a **SERIALIZABLE** Prisma transaction with retry handling. This decision directly targets overselling, duplicate payment callbacks, and concurrent tier capacity changes.

HMAC-signed QR tickets

Tickets embed a payload signed with **HMAC-SHA256**. The scanner verifies the signature using timing-safe comparison. This is stronger than trusting a plain ticket ID because users cannot forge event/order/ticket identities without the secret key.

Flexible scanner parser

The verify route accepts several QR formats because real camera libraries may return raw text, encoded text, URL query data, or JSON. This is a pragmatic reliability choice for venue operations.

Layered file storage

Uploads and PDFs can use **R2**, **Cloudinary**, generic **S3**, or local disk. This makes the app deployable in different environments. The trade-off is more URL parsing, deletion, signed access, and fallback code.

Optional Redis queue

The project prefers **BullMQ** for production ticket/email workloads but keeps a fallback path. This avoids blocking local development or low-cost deployments on Redis, while still allowing scalable background processing later.

Team roles instead of only global admin

TeamMember roles let organizers delegate scanning, management, analytics, and edit access per event. This is closer to real event operations than a single admin account. The trade-off is more authorization paths to test.

Certificate templates as PDF overlays

The certificate designer stores field coordinates and uses **pdf-lib** to overlay names/events/dates onto uploaded PDFs. This is more flexible than generating one fixed certificate design, but it requires authenticated template proxying and normalized coordinate handling.

Frontend lazy loading

Admin and public pages are loaded with **React.lazy** and **Suspense**. That keeps initial load lighter for a broad SPA. The trade-off is needing good loading states and build-time chunk behavior.

Alternatives And Why The Current Choices Make Sense

- **Next.js** could consolidate frontend and backend, but this app already has an Express API with raw webhook parsing, upload middleware, Prisma, queues, and scanner endpoints. The split API is easier to reason about for these backend-heavy flows.
- **MongoDB** could store dynamic forms easily, but the core domain is relational: one event has many orders, registrations, tickets, discounts, team members, polls, and reminders. PostgreSQL fits those constraints better.
- **Server-rendered forms** would simplify initial page load, but the product needs highly interactive form building, payment state, tier selection, discounts, and scanner behavior in the browser.
- **Always-on queue workers** would be cleaner operationally, but the optional Redis design is useful for student/demo environments and deployments where Redis is not configured yet.

5. Deep Dive - Hardest Problems

Question: Payment correctness and idempotency

Answer: The hardest correctness problem is accepting multiple payment confirmation paths without issuing duplicate tickets or overselling capacity. The code centralizes completion in **order-completion.service.js**, checks whether an order is already paid, updates the order and registration together, increments tier and discount counters, and retries serializable transaction conflicts.

Concept tested: Technical challenge and implementation detail

Question: Capacity and tier oversell prevention

Answer: Capacity exists at both event and ticket-tier levels. The completion transaction checks ticket tier availability, increments **soldCount**, and uses transaction isolation to reduce race conditions. This matters when two users pay around the same time for the last available ticket.

Concept tested: Technical challenge and implementation detail

Question: QR forgery resistance

Answer: A ticket ID alone is not enough because it can be guessed or copied. The QR payload includes ticket, event, order, registration, issue time, and HMAC signature. Verification uses **crypto.timingSafeEqual** to avoid leaking signature comparison timing.

Concept tested: Technical challenge and implementation detail

Question: Venue scanner reliability

Answer: The scanner must work under noisy real-world conditions. The backend accepts several QR encodings, supports manual entry, dedupes scanner requests, rejects duplicate scans, and uses atomic update predicates for check-in and checkout state.

Concept tested: Technical challenge and implementation detail

Question: PDF generation and delivery

Answer: Tickets and certificates are generated server-side and need durable storage. The code handles poster fetching, QR rendering, PDFKit ticket layout, R2 upload, Cloudinary fallback, local fallback, signed or proxied access, and on-demand PDF generation for order downloads.

Concept tested: Technical challenge and implementation detail

Question: Dynamic form validation

Answer: Each event can define a different form. The frontend renders fields dynamically, but the backend still needs to enforce required fields, email formats, option values, and safe response shape through **AJV**. This prevents trusting client-side form logic.

Concept tested: Technical challenge and implementation detail

Question: Role-based event access

Answer: Global organizer access is not enough. Admin routes, team routes, scanner routes, and certificate routes must check whether the caller owns the event or has an accepted team role with the required capability.

Concept tested: Technical challenge and implementation detail

Question: Operational fallback behavior

Answer: The system is designed to work in partial infrastructure states. Redis can be missing, Cloudinary can be absent, R2 can be unconfigured, Twilio can be disabled, and SMTP can fail. The code often logs and degrades instead of crashing the main registration flow.

Concept tested: Technical challenge and implementation detail

Question: Certificate template proxying

Answer: Certificate previews cannot always load raw Cloudinary or private R2 URLs in an iframe with auth headers. The backend therefore proxies templates and the frontend creates blob URLs for preview, which avoids leaking storage credentials and keeps protected documents viewable.

Concept tested: Technical challenge and implementation detail

Question: Analytics from normalized data

Answer: Admin analytics are computed across registrations, orders, tickets, tiers, discounts, and check-in timestamps. The challenge is presenting business metrics while preserving normalized records for payments, attendee details, and ticket state.

Concept tested: Technical challenge and implementation detail

Concurrency, Latency, Correctness, And Scale

- **Concurrency:** serializable payment completion and atomic check-in update predicates protect the most sensitive state transitions.
- **Latency:** ticket generation, email, and SMS work can be deferred through BullMQ or fallback background execution so payment verification can return quickly.
- **Correctness:** payment signatures, webhook raw body verification, HMAC QR validation, dynamic form validation, and role middleware protect trust boundaries.

- **Scale:** the app can move ticket generation and email delivery into Redis-backed workers, keep PDFs in object storage, serve the frontend statically, and scale the Express API separately from the SPA.

ML/AI Applicability

No **ML** or **AI** components are implemented in this repository. There are no model files, training loops, feature pipelines, embeddings, inference endpoints, or evaluation metrics. Because of that, a dedicated ML/AI section is intentionally omitted.

7. Interview Questions and Answers

Question: Walk me through the architecture of Occasio from the public event page to ticket delivery.

Answer: A public user loads the **React/Vite** SPA, which fetches event data, form schema, tiers, reviews, polls, and speakers through **Axios**. The backend is **Express** with route modules and **Prisma** over **PostgreSQL**. Registration data is validated against the dynamic form schema, then the backend creates a registration and order. Paid orders go through **Razorpay** or **PhonePe**. After verification or webhook confirmation, the order-completion service updates the order, registration, discounts, ticket tier counters, and queues ticket generation. The ticket service signs a QR payload, renders a PDF with **PDFKit**, stores it in **R2/Cloudinary/local storage**, and sends email/SMS.

Concept tested: End-to-end system design, component boundaries, and data flow.

Question: Why is PostgreSQL with Prisma a good fit for this project instead of a document database?

Answer: The core entities are strongly relational: users own events, events have forms, registrations, orders, tickets, discounts, polls, reviews, team members, tiers, speakers, and reminders. Prisma models those relations, enums, indexes, and migrations explicitly. The project still gets JSON flexibility through PostgreSQL JSONB fields for form schema, form responses, payment data, ticket style, certificate config, and discount snapshots. A pure document database would make dynamic forms easy but would make transactional payment completion, joins for analytics, and referential integrity harder.

Concept tested: Database modeling, trade-offs, and schema reasoning.

Question: How does the code prevent duplicate tickets or inconsistent payment state?

Answer: The key is the **completePaidOrder** service. Razorpay verification, PhonePe verification, free/RSVP completion, and webhooks all flow into one completion path. It runs a serializable Prisma transaction, checks whether the order is already paid, validates ticket tier availability, increments sold counts and discount usage, updates order and registration state, and enqueues ticket generation. It also retries Prisma transaction conflicts. This makes the payment state transition idempotent and reduces duplicated business logic across routes.

Concept tested: Idempotency, transactions, and payment correctness.

Question: How are QR tickets secured?

Answer: Tickets are not just random IDs. The QR utility builds a payload containing ticketId, eventId, orderId, registrationId, issuedAt, and a signature. The signature is generated with **HMAC-SHA256** using a server secret. Verification recomputes the signature and compares it with **timingSafeEqual**. The scanner route then checks ticket existence, event match, registration/payment status, revoked state, expiration, duplicate check-in state, and caller authorization. This makes forged or modified QR payloads fail verification.

Concept tested: Applied security, tamper resistance, and scanner trust boundaries.

Question: What validation patterns are used across the backend?

Answer: The backend uses multiple layers. Auth routes use **express-validator** for email/password/name checks. Dynamic registration forms use **AJV** with format validation because the schema is stored per event. Payment routes verify provider signatures. Admin routes validate event payloads, ticket style, team roles, certificate configs, discounts, and file upload types. Prisma constraints and enums protect database state. This layered approach is necessary because the frontend renders flexible forms that cannot be trusted alone.

Concept tested: Input validation, schema validation, and trust boundaries.

Question: How does the system handle background work and what happens if Redis is missing?

Answer: When Redis is configured and reachable, **BullMQ** queues process ticket generation and email sending. The worker creates the ticket PDF, stores it, sends the email, and may trigger SMS notification. If Redis is unavailable, the service falls back to **setImmediate** background execution. This keeps the registration/payment flow usable in local development or simple deployments, while still allowing production deployments to move expensive work to queues.

Concept tested: Async architecture, graceful degradation, and production readiness.

Question: Explain the event team permission model.

Answer: Users can be global admins/organizers, event owners, or invited team members. The backend has authentication middleware, organizer/admin guards, and event-access checks. Team roles include **SUPER_MANAGER**, **MANAGER**, **SCANNER**, and **STAFF**. The team routes expose only event lists, attendee lists, stats, check-in, checkout, and edit/form links depending on role and accepted invitation state. This is more realistic than one admin password because events often need temporary venue staff.

Concept tested: Authorization design and least-privilege access.

Question: How does the ticket scanning flow handle real-world QR input variability?

Answer: The ticket verification route accepts direct ticket IDs, JSON objects, JSON strings, URL-encoded JSON, base64 JSON, URL query strings, and raw payloads. This matters because scanner libraries and mobile cameras can return different formats. The route normalizes the input before validating the HMAC signature and ticket state. The frontend also supports manual entry, which is important when a camera fails or an attendee presents a damaged ticket.

Concept tested: Robust API design and operational resilience.

Question: What are the main performance risks in this codebase?

Answer: The heavy operations are PDF generation, poster fetching, email/SMS delivery, analytics queries, certificate overlays, and scanner bursts at venue entry. The code mitigates some of this by lazy-loading frontend routes, deferring ticket/email work through BullMQ or background tasks, storing generated PDFs in object storage, and using atomic updates for scans. To scale further, I would run dedicated queue workers, add pagination and indexes for large attendee lists, cache public event pages, and precompute some analytics.

Concept tested: Performance bottlenecks, latency management, and scaling strategy.

Question: How are certificates implemented?

Answer: The admin can upload a certificate PDF template and map fields in the frontend using a **react-pdf** preview. Field coordinates are normalized, saved in event certificate config, and later used by the backend certificate service. The service loads the template from data URL, R2, Cloudinary, HTTP, or local storage, then uses **pdf-lib** to draw attendee and event data onto the template. A protected backend proxy is used for template preview because private object URLs or Cloudinary raw URLs may require signed access.

Concept tested: Document generation, browser preview, and secure storage access.

Question: How would you scale Occasio for a large event with thousands of attendees?

Answer: I would keep the static frontend on Vercel/CDN, scale the Express API horizontally, move ticket generation and email/SMS into dedicated BullMQ workers, use managed Redis, store all PDFs in R2, add database indexes for eventId/status/checkedInAt, paginate attendees, and rate-limit scanner endpoints per event/team. For entry latency, I would keep scanner responses minimal, use atomic database updates, and possibly maintain a short-lived cache of valid ticket metadata while still writing final check-in state to PostgreSQL.

Concept tested: Scalability, deployment architecture, and operational planning.

Question: What would you change if you had more time?

Answer: I would formalize automated tests around payment webhooks, order completion races, team authorization, scanner duplicate scans, and certificate generation. I would extract long admin routes into smaller controllers/services, add OpenAPI documentation, make Redis workers deploy as a separate process, complete or remove placeholder wallet/Stripe paths, add observability around queue failures, and make analytics queries easier to evolve. The existing foundation is strong, but those changes would improve maintainability and production confidence.

Concept tested: Self-review, maintainability, and roadmap judgment.

8. Follow-up Questions

Question: If two users buy the last ticket tier slot at the same time, what protects the system?

Answer: The completion service uses a serializable transaction, checks tier availability inside the transaction, increments sold count during completion, and retries transaction conflicts. The strongest improvement would be a conditional update on soldCount with a capacity predicate, but the current design already identifies completion as the correct contention point.

Concept tested: Race-condition handling.

Question: Why is raw body parsing required for the payment webhook?

Answer: Webhook signatures are computed over the exact raw request payload. If Express JSON parsing changes whitespace or object ordering before verification, the signature check can fail or become meaningless. The server mounts raw parsing for webhook routes before normal JSON body parsing.

Concept tested: Webhook security.

Question: Why not generate tickets synchronously before returning from payment verification?

Answer: PDF generation, QR rendering, storage upload, and email delivery can be slow or flaky. Returning after the order is safely marked paid keeps the payment UX responsive. The generated ticket can be completed by BullMQ or fallback background work, and downloads can also generate PDFs on demand.

Concept tested: Latency and user experience.

Question: How does the app support both ticketed and RSVP events?

Answer: The event model has an **EventType** enum. Registration pricing logic treats RSVP/free events as zero-amount flows and completes them without a payment provider. Ticketed events create payment orders and wait for provider verification before marking registration paid/confirmed and issuing tickets.

Concept tested: Business rule modeling.

Question: Where are secrets handled and what should not be committed?

Answer: Backend and frontend example env files document required values, but actual secrets belong in deployment environments: JWT secret, database URL, Razorpay keys, PhonePe credentials, SMTP credentials, R2/S3 keys, Cloudinary secrets, Twilio keys, and VAPID private key. The PDF intentionally does not include live .env values.

Concept tested: Configuration and secret hygiene.

Question: What is the biggest maintainability concern?

Answer: The broadest concern is route size and feature concentration, especially in admin event control and admin backend routes. The domain has grown to include payments, uploads, certificates, teams, analytics, polls, reminders, and tickets. Splitting controllers, services, validators, and response mappers would make the code easier to test and evolve.

Concept tested: Maintainability and refactoring priorities.

9. Quick Reference Card

Elevator Pitch

Occasio is a production-oriented event management and booking platform built with **React**, **Express**, **Prisma**, and **PostgreSQL**. It lets organizers publish events, build custom registration forms, collect payments, issue HMAC-secured QR ticket PDFs, run team-based venue check-in, manage discounts and waitlists, view analytics, and send certificates. The most important engineering work is around payment correctness, dynamic form validation, secure ticket verification, object-storage-backed PDFs, and graceful queue/storage fallbacks.

5 Most Impressive Technical Facts

- Payment completion is centralized in a service that uses **serializable Prisma transactions** and retry handling to coordinate orders, registrations, tickets, discounts, and tier capacity.
- Tickets use **HMAC-SHA256 signed QR payloads** plus timing-safe verification, not just plain ticket IDs.
- The platform supports **dynamic event-specific registration forms** rendered by React and validated by AJV on the backend.
- Ticket and certificate PDFs are generated server-side with **PDFKit** and **pdf-lib**, then stored through R2/Cloudinary/S3/local fallback paths.
- Venue operations include team role delegation, camera/manual scanning, check-in/check-out state, analytics, and duplicate-scan protection.

3 Things That Make It Stand Out From Typical Student Projects

- It models the full event lifecycle: discovery, form building, payment, ticketing, venue scanning, analytics, reminders, reviews, polls, waitlists, and certificates.
- It handles real production concerns: webhook signatures, QR tamper resistance, object storage, optional queues, CORS/rate limiting, deployment configs, and secret-based integrations.
- It has real operational roles and workflows rather than only CRUD screens: organizer dashboards, team member access, scanner views, certificate dispatch, and financial reporting.

One-Minute Technical Walkthrough

Start with the public React app. It loads published events and a dynamic registration form. The Express backend validates the form, creates a registration and payment order, and hands off payment to Razorpay or PhonePe. When the payment is verified or the webhook arrives, a serializable transaction marks the order and registration paid, updates tier capacity and discounts, and schedules ticket generation. The ticket service signs a QR payload, generates a PDF, stores it in R2 or a fallback provider, and sends the attendee an email. At the venue, scanner users submit the QR payload; the backend verifies the HMAC signature, checks event access and ticket state, and atomically marks the ticket checked in.

Files And Evidence Used

- **Docs:** README.md, QUICKSTART.md, backend/README.md, frontend/README.md, and Event Management System Doc.pdf.
- **Backend:** server setup, config, Prisma schema/migrations, middleware, route modules, services, utilities, scripts, and node:test coverage.

- **Frontend:** App routing, auth/theme contexts, API client, public pages, admin pages, scanner/team pages, components, hooks, Tailwind config, Vite config, and PWA setup.
- **Deployment/config:** Azure workflow, Vercel config, startup script, package locks, and environment examples.